Documentation – Slime Mold Algorithm appliqué au TSP

Quentin Beauquier

1er avril 2026

Table des matières

Nomenclature **2**

1 Définition **4**
1.1 Principe de base 4
1.2 Application 4

2 Méthode **5**
2.1 Mise à jour de la position 5
2.2 Poids des agents 5

3 Analogie biologique **6**

4 Cas d'application connus **7**

5 Exemple de code **8**

6 Benchmark TSP **10**
6.1 Métriques 10
6.2 Résultats 10

Nomenclature

Symboles mathématiques

Symbole	Définition
X_i	Tournée (liste ordonnée de sommets) de l'agent i
X_b	Tournée de la meilleure solution trouvée
X_A, X_B	Tournées de deux agents sélectionnés par tirage pondéré
$W(X_i)$	Poids attribué à l'agent X_i, fonction de son rang de fitness
$S(X_i)$	Valeur de fitness (coût total de la tournée) de l'agent X_i
bF	Meilleure fitness de l'itération courante (*best fitness*)
wF	Pire fitness de l'itération courante (*worst fitness*)
p	Probabilité de croisement : $p = \tanh(\lvert S(X_i) - bF \rvert)$
r	Nombre aléatoire tiré uniformément dans $[0, 1]$
n	Nombre total d'agents dans la population
t	Indice de l'itération courante
T	Nombre maximal d'itérations

Termes techniques

Terme	**Définition**
Métaheuristique	Méthode d'optimisation générique, souvent stochastique, applicable à une large classe de problèmes sans garantie d'optimalité
Myxomycète	Organisme unicellulaire capable de former des réseaux de transport efficaces ; *Physarum polycephalum* en est l'espèce de référence
Fitness	Fonction objectif évaluant la qualité d'une solution ; dans le cas du TSP, il s'agit du coût total de la tournée (somme des distances euclidiennes)
Agent	Entité de la population représentant une solution candidate (une tournée complète), analogue à un filament du myxomycète
Tournée	Permutation des sommets du graphe définissant un cycle hamiltonien
Exploitation	Phase où l'algorithme affine les solutions proches de la meilleure solution connue
Exploration	Phase où l'algorithme diversifie la recherche dans des régions inexplorées de l'espace des solutions
OX Crossover	Opérateur de croisement ordonné (*Order Crossover*) préservant l'ordre relatif des éléments entre deux solutions parentes
Random Swap	Opérateur de mutation échangeant aléatoirement deux sommets dans une tournée
2-opt	Heuristique d'amélioration locale consistant à supprimer deux arêtes d'une tournée et à les reconnecter de manière à réduire le coût total
TSP	Problème du voyageur de commerce (*Travelling Salesman Problem*) : déterminer le cycle hamiltonien de coût minimal dans un graphe complet
Gap	Écart relatif entre le coût d'une solution obtenue et le coût de la solution optimale connue
Convergence	État atteint lorsque l'algorithme ne parvient plus à améliorer significativement la solution au fil des itérations
Arrêt anticipé	Mécanisme interrompant l'exécution lorsque ni la phase de 2-opt ni la phase d'exploration ne produisent d'amélioration

Chapitre 1

Définition

Le *Slime Mold Algorithm* (SMA) est une métaheuristique d'optimisation inspirée du comportement du myxomycète *Physarum polycephalum*. Proposé par Li et al. (2020), il modélise la capacité de cet organisme à former des réseaux de transport efficaces en explorant son environnement à la recherche de nourriture.

1.1 Principe de base

Le *SMA* simule le comportement du myxomycète. Celui-ci se déploie de manière stochastique afin d'explorer son environnement et de localiser des sources nutritives, en établissant des canaux permettant leur acheminement. Lorsqu'il rencontre une source de nourriture, le myxomycète renforce le chemin le plus court afin d'optimiser sa consommation d'énergie et supprime les canaux inutiles.

1.2 Adaptation au TSP

L'algorithme original de Li et al. opère dans un espace continu. Notre implémentation l'adapte au problème du voyageur de commerce, qui relève de l'optimisation combinatoire. Les mises à jour de position continues sont remplacées par des opérateurs discrets sur des permutations :

- Le **croisement ordonné** (OX) se substitue à la mise à jour vectorielle de la phase d'exploitation ;
- La **mutation par échange aléatoire** (*random swap*) remplace la contraction de la phase d'exploration ;
- L'optimisation locale **2-opt** est intégrée de manière cyclique afin d'affiner les tournées obtenues.

Chapitre 2

Architecture

L'implémentation se compose de quatre modules principaux.

2.1 Graph

La classe `Graph` encapsule le problème TSP. Elle stocke la liste des coordonnées des villes et fournit les méthodes suivantes :

- `distance(i, j)` : distance euclidienne entre les villes i et j ;
- `turn_cost(turn)` : coût total d'une tournée (somme des distances consécutives, y compris le retour à la ville de départ) ;
- `random(nb_cities)` : génération d'une instance aléatoire ;
- `plot(turn, ...)` : visualisation de la tournée et de la courbe de convergence.

2.2 Agent

La classe `Agent` représente une solution candidate. Chaque agent possède trois attributs :

- `turn` : la tournée courante (permutation des indices de villes) ;
- `cost` : le coût de cette tournée, calculé à l'initialisation via `graph.turn_cost` ;
- `weight` : le poids utilisé lors de la sélection pondérée (initialisé à 1,0).

La méthode de classe `Agent.random(graph)` crée un agent dont la tournée est une permutation aléatoire de l'ensemble des villes.

2.3 Opérateurs

Trois opérateurs sont définis dans le module `operators` :

2.3.1 Random Swap (mutation)

L'opérateur `random_swap` sélectionne deux positions aléatoires dans la tournée et permute les villes correspondantes. Il s'agit de l'opérateur d'exploration, appliqué lorsqu'un agent est jugé trop éloigné de la meilleure solution.

2.3.2 Order Crossover – OX (croisement)

L'opérateur `ox_crossover` effectue un croisement ordonné entre deux parents. Il sélectionne un segment aléatoire $[a, b)$ du premier parent, le copie dans l'enfant, puis complète les positions restantes avec les villes du second parent, en préservant leur ordre relatif d'apparition.

Dans notre implémentation, le croisement est appliqué en cascade :

$$X' = \mathrm{OX}\big(X_b,\ \mathrm{OX}(X_A,\ X_B)\big) \tag{2.1}$$

Le croisement intermédiaire $\mathrm{OX}(X_A, X_B)$ recombine deux agents sélectionnés par tirage pondéré, puis le résultat est croisé avec la meilleure solution X_b, orientant ainsi la recherche vers les régions les plus prometteuses.

2.3.3 2-opt (amélioration locale)

L'opérateur `two_opt` parcourt toutes les paires d'arêtes $(i, i{+}1)$ et $(j, j{+}1)$ de la tournée. Si la suppression de ces deux arêtes et leur reconnexion croisée diminue le coût total, le segment $[i{+}1, j]$ est inversé. L'opérateur effectue au plus `max_passes` passes complètes et s'arrête prématurément si aucune amélioration n'est trouvée lors d'une passe.

2.4 Population

La classe `Population` orchestre l'ensemble de l'algorithme. Elle gère la collection d'agents et expose deux méthodes principales : `step` (une itération) et `run` (boucle complète avec arrêt anticipé).

Chapitre 3

Méthode

3.1 Calcul des poids

À chaque itération, les agents sont triés par coût croissant. Le poids $W(X_i)$ de chaque agent est calculé en fonction de son rang :

$$W(X_i) = \begin{cases} 1 + r \cdot \log\left(\dfrac{bF - S(X_i)}{bF - wF} + 1\right) & \text{si } i < \lfloor n/2 \rfloor \\ 1 - r \cdot \log\left(\dfrac{bF - S(X_i)}{bF - wF} + 1\right) & \text{si } i \geq \lfloor n/2 \rfloor \end{cases} \tag{3.1}$$

où $r \sim \mathcal{U}(0, 1)$. Les agents de la moitié supérieure (meilleures solutions) reçoivent un poids supérieur à 1, renforçant leur probabilité d'être sélectionnés lors du croisement. Les agents de la moitié inférieure reçoivent un poids inférieur à 1. Si $bF = wF$ (population homogène), les poids demeurent inchangés.

3.2 Mise à jour des positions

Pour chaque agent, la probabilité de croisement est calculée :

$$p = \tanh\big(|S(X_i) - bF|\big) \tag{3.2}$$

Plus un agent est éloigné de la meilleure solution, plus p est élevé, et plus la probabilité de croisement augmente. La mise à jour s'effectue selon :

$$X_i(t+1) = \begin{cases} \mathrm{OX}\big(X_b,\ \mathrm{OX}(X_A,\ X_B)\big) & \text{si } r < p \quad \text{(exploitation)} \\ \mathrm{random_swap}(X_i) & \text{sinon} \quad \text{(exploration)} \end{cases} \tag{3.3}$$

où X_A et X_B sont sélectionnés par tirage pondéré (sans remplacement entre eux) proportionnellement à leur poids W.

La nouvelle tournée n'est acceptée que si son coût est strictement inférieur au coût courant de l'agent (**acceptation gloutonne**).

3.3 Phases cycliques

L'exécution est structurée en **cycles** de longueur `phase_length` (par défaut 15) itérations. Chaque cycle se décompose en deux phases :

1. **Phase d'exploration** (premières $\lceil (1 - \texttt{opt_ratio}) \times \texttt{phase_length} \rceil$ itérations) : seuls les opérateurs de croisement et de mutation sont appliqués.
2. **Phase d'optimisation locale** (dernières $\lfloor \texttt{opt_ratio} \times \texttt{phase_length} \rfloor$ itérations, par défaut 25 %) : l'opérateur 2-opt est appliqué en complément à chaque agent, avec au plus `max_opt_passes` passes (par défaut 5).

Cette alternance permet de diversifier la recherche avant de l'affiner localement, évitant ainsi une convergence prématurée.

3.4 Arrêt anticipé

Le mécanisme d'arrêt anticipé surveille la stagnation de l'algorithme :

1. À l'entrée de chaque phase 2-opt, le meilleur coût courant est enregistré (*snapshot*).
2. Si, à l'issue de la phase 2-opt, le meilleur coût n'a pas diminué (phase jugée *stale*), un indicateur est levé.
3. Si, lors de la phase d'exploration suivante, le meilleur coût demeure inchangé, l'algorithme conclut que ni l'exploitation ni l'exploration ne produisent d'amélioration et met fin à l'exécution.

Chapitre 4

Analogie biologique

Le myxomycète explore son environnement en déployant ses filaments vers les sources de nourriture. Les filaments menant aux meilleures sources se renforcent, les autres s'atrophient.

Dans l'algorithme, chaque agent représente un filament. Les agents proches de bonnes solutions voient leur poids W augmenter (phase d'exploitation), tandis que les agents éloignés explorent davantage (phase d'exploration). La probabilité $p = \tanh(|S(X_i) - bF|)$ traduit ce mécanisme : plus un filament est éloigné d'une source nutritive, plus il est susceptible d'être reconfiguré par croisement avec des filaments mieux positionnés.

Chapitre 5

Cas d'application connus

— Optimisation de fonctions continues (benchmark CEC)
— Sélection de caractéristiques en apprentissage automatique
— Problèmes d'ingénierie (conception mécanique, réseaux électriques)
— Problème du voyageur de commerce (TSP)
— Ordonnancement de tâches

Référence principale : Li, S., Chen, H., Wang, M., Heidari, A. A., & Mirjalili, S. (2020). *Slime mould algorithm : A new method for stochastic optimization*. Future Generation Computer Systems, 111, 300–323.

Chapitre 6

Hyperparamètres

Paramètre	**Description**	**Défaut**	**Unité**
`nb_agents`	Taille de la population	40	agents
`T`	Nombre maximal d'itérations	200	itérations
`phase_length`	Longueur d'un cycle exploration/2-opt	15	itérations
`opt_ratio`	Proportion du cycle dédiée au 2-opt	0,25	–
`max_opt_passes`	Passes maximales de 2-opt par itération	5	passes

Table 6.1 – Hyperparamètres de l'algorithme et valeurs par défaut

Chapitre 7

Exemples de code

7.1 Agent

```
class Agent:
    def __init__(self, graph, turn):
        self.graph = graph
        self.turn = turn
        self.weight = 1.0
        self.cost = self.graph.turn_cost(turn)

    @classmethod
    def random(cls, graph):
        turn = list(range(len(graph.cities)))
        random.shuffle(turn)
        return cls(graph, turn)
```

7.2 Calcul des poids

```
def update_weights(self):
    n = len(self.agents)
    self.agents.sort(key=lambda a: a.cost)
    bF = self.best.cost
    wF = self.worst.cost
    if bF == wF:
        return
    for i in range(0, n):
        s = self.agents[i].cost
        calc = np.log((bF - s) / (bF - wF) + 1)
```

```
        if i < n / 2:
            self.agents[i].weight = 1 + random.uniform(0,
                1) * calc
        else:
            self.agents[i].weight = 1 - random.uniform(0,
                1) * calc
```

7.3 Itération principale

```
def step(self, t=0, phase_length=20, opt_ratio=0.25,
   max_opt_passes=5):
    self.update_weights()
    X_b = self.best.turn
    bF = self.best.cost

    for agent in self.agents:
        p = np.tanh(np.abs(agent.cost - bF))
        if random.uniform(0, 1) < p:
            X_A = random.choices(
                self.agents,
                weights=[a.weight for a in self.agents],
                   k=1)[0]
            remaining = [a for a in self.agents if a is
               not X_A]
            remaining_weights = [a.weight for a in
               remaining]
            X_B = random.choices(
                remaining, weights=remaining_weights, k
                   =1)[0]
            new_turn = ox_crossover(X_b, ox_crossover(X_A
               .turn, X_B.turn))
        else:
            new_turn = random_swap(agent.turn)

        new_cost = self.graph.turn_cost(new_turn)
        if new_cost < agent.cost:
            agent.turn = new_turn
            agent.cost = new_cost

        if t % phase_length >= phase_length * (1 -
           opt_ratio):
```

```
            agent.turn = two_opt(
                agent.turn, self.graph, max_passes=
                    max_opt_passes)
            agent.cost = self.graph.turn_cost(agent.turn)
```

7.4 Opérateurs

```
def random_swap(turn):
    tmp = turn[:]
    a, b = random.randint(0, len(turn)-1), random.randint
        (0, len(turn)-1)
    tmp[a], tmp[b] = tmp[b], tmp[a]
    return tmp

def ox_crossover(parent1, parent2):
    n = len(parent1)
    a, b = sorted([random.randint(0, n-1), random.randint
        (0, n-1)])
    enfant = [None] * n
    enfant[a:b] = parent1[a:b]
    placed = set(parent1[a:b])
    remaining = [v for v in parent2 if v not in placed]
    j = 0
    for i in range(n):
        if enfant[i] is None:
            enfant[i] = remaining[j]
            j += 1
    return enfant

def two_opt(turn, graph, max_passes=5):
    n = len(turn)
    for _ in range(max_passes):
        improved = False
        for i in range(0, n-2):
            for j in range(i+2, n-1):
                gain = (graph.distance(turn[i], turn[i
                    +1])
                        + graph.distance(turn[j], turn[j
                            +1])
                        - graph.distance(turn[i], turn[j])
```

```
                        - graph.distance(turn[i+1], turn[j
                            +1]))
                    if gain > 0:
                        turn[i+1:j+1] = turn[i+1:j+1][::-1]
                        improved = True
        if not improved:
            break
    return turn
```

7.5 Boucle d'exécution avec arrêt anticipé

```
def run(self, T, phase_length=15, opt_ratio=0.25,
        max_opt_passes=5, on_step=None):
    opt_phase_start = int(phase_length * (1 - opt_ratio))
    best_before_opt = None
    best_after_opt = None
    opt_was_stale = False
    history = []

    for t in range(T):
        cycle_pos = t % phase_length
        is_opt_phase = cycle_pos >= opt_phase_start
        phase = "2-opt" if is_opt_phase else "exploration
            "

        self.step(t, phase_length=phase_length,
                  opt_ratio=opt_ratio, max_opt_passes=
                     max_opt_passes)
        best_cost = self.best.cost
        history.append(best_cost)

        if on_step:
            on_step(t, self, history, phase, stopped=
               False)

        # Entrée en 2-opt : snapshot du meilleur coût
        if cycle_pos == opt_phase_start:
            best_before_opt = best_cost

        # Pendant le 2-opt : vérifier la stagnation
        if is_opt_phase:
```

```
            opt_was_stale = (best_before_opt is not None
                             and best_cost >=
                                best_before_opt)
            best_after_opt = best_cost

        # Exploration suivante : si 2-opt était stale
        # et aucune amélioration -> arrêt
        if not is_opt_phase and opt_was_stale:
            if (best_after_opt is not None
                    and best_cost >= best_after_opt):
                if on_step:
                    on_step(t, self, history, phase,
                       stopped=True)
                break

    return self.best.cost, len(history), history, self.
       best.turn
```

Chapitre 8

Benchmark TSP

8.1 Infrastructure de benchmark

L'implémentation fournit deux outils de benchmark :

— `bench_single.py` : évaluation sur un ou plusieurs fichiers CSV (fusionnés le cas échéant), avec visualisation en temps réel optionnelle et paramétrage via la ligne de commande ;

— `benchmark.py` : campagne comparative sur plusieurs instances de tailles variées (20, 50, 100 villes issues de TSPLIB, plus une instance *world*).

Les deux outils exécutent l'algorithme avec plusieurs tailles de population (10, 20, 40, 80 agents) et produisent des graphiques comparatifs (tournées, courbes de convergence, coût *vs.* temps).

8.2 Formats de données acceptés

— **Format TSP dataset** : colonne `city_coordinates` contenant une liste de tuples, et colonne `total_distance` pour la solution optimale connue ;

— **Format coordonnées** : colonnes `x`/`XCOORD.` et `y`/`YCOORD.` (instances Solomon, *world*).

8.3 Métriques

8.4 Résultats

Métrique	**Définition**
% réussite	Proportion de runs où le gap avec l'optimal est $\leq 1\,\%$
% non-détection	L'algorithme estime avoir convergé mais le gap réel est $> 1\,\%$
% fausse détection	L'algorithme s'arrête en estimant échouer alors que le gap est $\leq 1\,\%$
Gap moyen	Écart relatif moyen par rapport à l'optimal sur N exécutions
Écart-type du gap	Mesure de la robustesse sur N exécutions
Itérations	Nombre d'itérations avant convergence (ou arrêt anticipé)
Temps	Durée totale d'exécution (en secondes)

Table 8.1 – Définition des métriques de benchmark

Nœuds	**% réussite**	**% non-détection**	**% fausse détection**	**Temps (s)**
20	–	–	–	–
50	–	–	–	–
100	–	–	–	–
200	–	–	–	–
500	–	–	–	–
1 000	–	–	–	–

Table 8.2 – Benchmark SMA sur instances TSP (à compléter)